

Practica Repaso MC

Semaforos

Ejercicio 1

a_

```
sem mutexCola = 1;
cola cola;
sem esperaTerminal[P] = ((P) 0)

process Persona[id = 0 to P-1] {
    int aux;
    P(mutexCola)
    if (!libre) {
        push(cola, id);
        V(mutexCola)

        P(esperaTerminal[id]);
    }
    else {
        libre = true;
        V(mutexCola);
    }

    UsarTerminal();

    P(mutexCola);
    if (!cola.isEmpty()) {
        pop(cola, aux);
        V(esperaTerminal[aux]);
    }
    else {
        libre = true;
    }
}
```

```
V(mutexCola);  
}
```

b_

```
sem mutexCola = 1;  
cola cola;  
sem esperaTerminal[P] = ((P) 0)  
  
sem mutexColaTerminales = 1;  
cola colaTerminales;  
Terminal terminales[3];  
bool soyPrimero = true;  
  
process Persona[id = 0 to P-1] {  
  int aux, auxTerm;  
  P(mutexColaTerminales)  
  if (soyPrimero) {  
    soyPrimero = false;  
    push(colaTerminales, 0)  
    push(colaTerminales, 1)  
    push(colaTerminales, 2)  
  }  
  V(mutexColaTerminales)  
  
  P(mutexColaTerminales)  
  if (colaTerminales.isEmpty()) {  
    V(mutexColaTerminales)  
    P(mutexCola)  
    push(cola, id);  
    V(mutexCola)  
    P(esperaTerminal[id]);  
  } else  
    V(mutexColaTerminales);  
  
  P(mutexColaTerminales)  
  pop(colaTerminales, auxTerm);  
  V(mutexColaTerminales);  
}
```

```

UsarTerminal(terminales[auxTerm]);

P(mutexColaTerminales)
push(colaTerminales, auxTerm);
V(mutexColaTerminales);

P(mutexCola);
if (!cola.isEmpty()) {
    pop(cola, aux);
    V(esperaTerminal[aux]);
}
V(mutexCola);
}

```

Ejercicio 2

```

sem mutexCola = 1;
cola Transacciones;

sem mutexTerminados = 1;
int terminados = 0;

int resultadosTransacciones[10] = ((10) 0);
sem mutexResultadosTransacciones[10] = ((10) 1);

process Worker [id = 0 to 6] {
    Transaccion t;
    int valor;
    P(mutexCola);
    while (!colaTransacciones.isEmpty()) {
        pop(colaTransacciones, t)
        V(mutexCola);

        valor = Validar(t);

        P(mutexResultadosTransacciones[valor]);
    }
}

```

```

    resultadosTransacciones[valor]++;
    V(mutexResultadosTransacciones[valor])

    P(mutexColaTransacciones);
}
V(mutexCola);

P(mutexTerminados);
terminados++;
if (terminados == 7) {
    for int i = 0 to 9 {
        print(resultadosTransacciones[i])
    }
}
V(mutexTerminados);
}

```

Ejercicio 3

```

Cola colaMaquina;
bool libre = true;
int cantLatas = 100;

sem mutexLibre = 1;
sem mutexCola = 1;
sem esperaMaquina[U] = ((U) 0);
sem mutexLatas = 1;
sem despertarRepositor = 0;
sem esperandoRepositor = 0;

process Usuario [id = 0 to U-1]{
    int aux;
    P(mutexLibre)
    if (!libre){
        V(mutexLibre)
        P(mutexCola)
        push(colaMaquina. id);
    }
}

```

```

        V(mutexCola)

        P(esperaMaquina[id])
    } else {
        libre = false;
        V(mutexLibre)
    }

    P(mutexLatas)
    if (cantLatas == 0){
        V(mutexLatas)
        V(despertarRepositor)
        P(esperandoRepositor)
        P(mutexLatas)
    }
    cantLatas--;
    V(mutexLatas)

    P(mutexCola)
    if (!colaMaquina.isEmpty()){
        pop(colaMaquina, aux);
        V(mutexCola)
        V(esperaMaquina[aux])
    } else {
        V(mutexCola)
        P(mutexLibre)
        libre = true;
        V(mutexLibre)
    }
}

process repositor {
    while (true){
        P(despertarRepositor)
        P(mutexLatas)
        cantLatas = 100;
        V(mutexLatas)
        V(esperandoRepositor)
    }
}

```

```
}  
}
```

Monitores

Ejercicio 1

// Precondicion: la cola se ordena por prioridad y orden de llegada

```
monitor Maquina {  
  
    cola colaOrdenada;  
    esperando = 0;  
    cond esperaMaquina[N];  
    cond esperadAutoridad;  
  
    procedure solicitarMaquina(id: in int, tipo: in tipo)  
        push(colaOrdenada, id, tipo);  
        esperando++;  
  
        signal(esperaAutoridad);  
  
        wait(esperaMaquina[id]);  
  
        Votar();  
  
        signal(esperaAutoridad);  
    }  
  
    procedure darAccesoMaquina() {  
        int aux;  
        if (esperando > 0)  
            wait(esperaAutoridad);  
  
        pop(colaOrdenada, aux);
```

```

    esperando--;

    signal(esperaMaquina[aux]);

    wait(esperaAutoridad);
}
}

process Persona [id = 0 to N-1] {
    Maquina.solicitarMaquina(id, tipo);
}

process AutoridadDeMesa [] {
    for int i = 1 to N {
        Maquina.darAccesoMaquina()
    }
}

```

Ejercicio 2

```

monitor Barrera {
    cond esperaEquipos[5];
    int contadorEsperaEquipos[5] = ((5) 0);

    procedure Llegada(equipo: in int) {
        contadorEsperaEquipos[equipo]++;
        if (contadorEsperaEquipos[equipo] == 4)
            signal_all(esperaEquipos[equipo]);
        else
            wait(esperaEquipos[equipo]);
    }
}

monitor Contador {
    int contadorVentasEquipos[5] = ((5) 0);
    cond esperaEquipos[5];
}

```

```

int contadorEsperaEquipos[5] = ((5) 0);

procedure sumarVentas(equipo: in int; ventas: in int; ventasTotales: out i
nt)
    contadorVentasEquipos[equipo] += ventas;

    contadorEsperaEquipos[equipo]++;
    if (contadorEsperaEquipos[equipo] != 4)
        wait(esperaEquipos[equipo]);
    else
        signal_all(esperaEquipos[equipo]);

}

process VendedorAmbulante [id = 0 to 19] {
    int equipo;
    int ventas = 0;
    int ventasTotales;

    Barrera.llegada(equipo);

    realizarVentas(ventas);

    Contador.sumarVentas(equipo, ventas, ventasTotales);
}

```

Solucion alternativa con multiples monitores

```

monitor monitorEquipo[id = 0 to 4] {
    cond esperaEquipo;
    int contadorEsperaEquipo = 0;

    int contadorVentasEquipo = 0;

    procedure llegada( {
        contadorEsperaEquipo++;
        if (contadorEsperaEquipo == 4)
            signal_all(esperaEquipo);
    }
}

```



```

        contadorEsperaEquipo = 0;
    else
        wait(esperaEquipo);
    }

    procedure sumarVentas(ventas: in int; ventasTotales: out int)
        contadorVentasEquipo += ventas;

        contadorEsperaEquipo++;
        if (contadorEsperaEquipo != 4)
            wait(esperaEquipo);
        else
            signal_all(esperaEquipo);
    }

    process VendedorAmbulante [id = 0 to 19] {
        int equipo;
        int ventas = 0;
        int ventasTotales;

        moonitorEquipo[equipo].llegada();

        realizarVentas(ventas);

        monitorEquipo[equipo].sumarVentas(ventas, ventasTotales);
    }

```

Ejercicio 3

```

    process Escalador [id = 0 to 29]{
        PasoMontania.LlegadaPaso();
        PasoMontania.LiberarPaso();
    }

    Monitor PasoMontania {
        int esperando = 0;
        procedure LlegadaPaso (){

```

```
    if(!libre){
        esperando++;
        wait(esperandoPaso);
    } else {
        libre = false;
    }
}

procedure LiberarPaso(){
    if (esperando > 0){
        esperando--;
        signal(esperandoPaso)
    } else {
        libre = true;
    }
}
}
```